

Assignment 4: JVM Code Generation for MiniRust

Version: v1.0

1. OBJECTIVES

- Understand how block-structured language features (functions, local variables, structs, loops with break/continue, conditional branches, switch statements, expressions) compile to JVM bytecode.
 - Learn stack-oriented code generation techniques and local variable slot management.
 - Implement structure compilation by generating separate public classes on the JVM.
 - Build a robust compilation and execution testing harness using the Jasmin assembler and JVM.
 - Create a comprehensive suite of unit tests to verify your implementation.
-

2. REQUIRED TASKS TO COMPLETE

- **Task 1: Study the AST Node definitions**
 - Read `src/utils/AST.py` to ensure you understand all the abstract syntax tree representations.
 - **Task 2: Implement CodeGen.py and Emitter.py**
 - Complete the class `CodeGenerator` in `src/codegen/CodeGen.py` (inheriting from `BaseVisitor`).
 - Complete the helper class `Emitter` in `src/codegen/Emitter.py` to generate correct JVM descriptors and Jasmin instructions.
 - Generate a main public class named `MiniRustProgram` containing all compiled static functions, and create a separate JVM class file for each `struct` defined in the program.
 - **Task 3: Write 100 codegen tests**
 - Provide 100 test cases in `test/test-codegen.py` (`test_1` to `test_100`) using the provided `CodeGenerator` helper.
 - Verify all aspects of variable assignments, scopes, loops, struct fields, functions, and control flow.
-

3. PREDEFINED STANDARD LIBRARY FUNCTIONS

Your code generator must support the following predefined standard library functions for console output:

- `print_i32(val: i32) -> void` / `println_i32(val: i32) -> void`
 - `print_f32(val: f32) -> void` / `println_f32(val: f32) -> void`
 - `print_bool(val: bool) -> void` / `println_bool(val: bool) -> void`
 - `print_string(val: string) -> void` / `println_string(val: string) -> void`
 - `println() -> void` (prints a newline)
-

4. PROJECT STRUCTURE

The repository layout for Assignment 4 must be structured as follows:

```
root/
├─ src/
│   └─ grammar/
```

```

|   |   |   | MiniRust.g4      # Student's ANTLR4 grammar file
|   |   |   | (generated files) # Generated by ANTLR (Lexer, Parser, etc.)
|   |   |   |
|   |   |   | astgen/
|   |   |   |   | ASTGenerator.py # Student's AST generator implementation (from
Assignment 2)
|   |   |   |   | checker/
|   |   |   |   |   | StaticChecker.py # Student's Static Checker implementation (from
Assignment 3)
|   |   |   |   | codegen/
|   |   |   |   |   | CodeGen.py      # Student's Code Generator class
|   |   |   |   |   | Emitter.py      # Student's Emitter helper class
|   |   |   |   |   | (provided helpers) # JasminCode.py, Frame.py, Io.py, etc.
|   |   |   |   | runtime/
|   |   |   |   |   | jasmin.jar      # Jasmin assembler (provided)
|   |   |   |   |   | io.java         # I/O support class (provided)
|   |   |   |   |
|   |   |   | test/
|   |   |   |   | TestUtils.py        # Testing utility class (provided)
|   |   |   |   | test-codegen.py     # Student's 100 codegen unit tests
|   |   |   | requirements.txt

```

5. EVALUATION CRITERIA & GRADING

- **Code Generation Correctness (100%):** Grade is based on passing the automated test cases run against your submitted files.
- **Test Cases Penalty:** Writing at least 100 unit tests in `test/test-codegen.py` is mandatory. A penalty of 10% of the total assignment grade will be deducted if `test/test-codegen.py` is missing or contains fewer than 100 test cases.

Test Distribution Table

The 100 tests in `test/test-codegen.py` must cover the following categories:

Category	Test Cases	Description
Basic Statements & Functions	test_1 - test_10	Printing primitives, basic operations, variable declarations, and simple function calls.
Switch Statement Control Flow	test_11 - test_15, test_41 - test_45	Basic switch branching, fallthroughs, switch-case break statements, and default clauses.
Loop Structures & Controls (While/For)	test_16 - test_25, test_35 - test_40	while loops and range-based for loops (for var in start..end), checking boundary and loop variables.
Loop Control Statements (Break/Continue)	test_26 - test_34, test_58 - test_67	break and continue statement behavior, including nested loop scenarios.
Struct Lifecycles & Field Accesses	test_46 - test_52, test_99	Struct instantiation, struct class member reads/writes, and passing structs as parameters.
Short-Circuit Logical Operations	test_53 - test_57	Boolean short-circuit evaluations (&& and `

Assignment Expressions & Chainings	test_68 - test_75	Right-associative assignments, assignments in control-flow conditions, and member assignments.
Relational & Operator Logic	test_76 - test_80	Relational comparison ops, logical NOT, and complex nested conditional branching.
Unary Expressions & Negations	test_81 - test_85	Unary minus, logical negations, and basic local operand modifications.
Floating-point & Coercions	test_86 - test_90	Floating-point arithmetic and implicit type coercions from i32 to f32.
Recursion & Scope Scenarios	test_91 - test_98, test_100	Factorial, fibonacci, mutual recursions, nested block variable shadowing, and FizzBuzz.

6. DELIVERABLES

Students must submit:

- `src/codegen/CodeGen.py`
- `src/codegen/Emitter.py`
- `test/test-codegen.py`

7. DEADLINE

- **Submission Deadline:** Refer to the [deadlines.json](#) Gist file for the exact due date for Assignment 4.

8. CHANGELOG

- First release under v1.0.